EMBEDDED SYSTEMS

Engineering Mastery Guide

100 Problems | 20 Projects | Edge AI & TinyML

Beginner to Advanced | EU Job Market Ready

Bare-Metal to RTOS | Hardware to Edge AI

Table of Contents

Right-click the TOC and select "Update Field" to refresh page numbers

Introduction to Embedded Systems.....3

Phase 1: C/C++ for Embedded (Problems 1-10).....4

Phase 2: Microcontroller Architecture (Problems 11-20).....7

Phase 3: Memory Management (Problems 21-30).....10

Phase 4: Interrupts and Timers (Problems 31-40).....13

Phase 5: Communication Protocols (Problems 41-55).....16

Phase 6: RTOS and Concurrency (Problems 56-70).....20

Phase 7: Low-Power Design (Problems 71-80).....24

Phase 8: Hardware Interfacing (Problems 81-90).....27

Phase 9: Edge AI & TinyML (Problems 91-100).....30

Projects 1-5: Foundation Projects.....33

Projects 6-10: Intermediate Projects.....36

Projects 11-15: Advanced Projects.....39

Projects 16-20: Expert Level Projects.....42

Hardware Recommendations.....45

EU Job Market Guide.....47

Introduction to Embedded Systems

Welcome to the world of embedded systems engineering. This guide will take you from complete beginner to a level where you can confidently apply for positions at top EU companies like Bosch, Siemens, STMicroelectronics, NXP, and automotive giants.

What Are Embedded Systems?

Embedded systems are specialized computing systems that perform dedicated functions within larger mechanical or electrical systems. Unlike general-purpose computers, embedded systems are designed to do specific tasks, often with real-time constraints, limited resources, and strict reliability requirements.

Why Embedded Systems?

- Every modern device contains embedded systems - from toasters to cars to spacecraft
- High demand in EU - especially automotive, industrial automation, IoT, and medical devices
- Combines software and hardware - perfect for engineers who love both
- Edge AI is revolutionizing the field - TinyML is the future

How to Use This Guide

- Start with simulations - no hardware needed for learning phase
- Use QEMU, Renode, or Proteus for microcontroller simulation
- For projects, hardware investment is recommended - detailed list provided
- Each problem builds on previous ones - do not skip
- Document everything in your engineering notebook

Simulation Tools You Will Need

- QEMU - ARM emulation (free, open source)
- Renode - Complete embedded simulation platform
- Proteus - Circuit and microcontroller simulation
- Keil MDK - ARM development (free for STM32)
- STM32CubeIDE - Free IDE for STM32 development
- PlatformIO - Cross-platform embedded development

Phase 1: C/C++ for Embedded Systems

Problems 1-10 | Focus: Embedded C, Bit Manipulation, Volatile, Const, Memory-Mapped I/O

What You Will Learn

Embedded C is different from desktop C. You will learn how to write code that directly controls hardware, manages memory at the byte level, and operates within strict resource constraints.

Problem 1: Bit Manipulation Fundamentals

Description: Write functions to: set a specific bit, clear a specific bit, toggle a bit, check if a bit is set, and extract a bit field from a 32-bit register. Test with various input values including edge cases.

What You Need to Learn:

- Bitwise operators: AND (&), OR (|), XOR (^), NOT (~), shift (<<, >>)
- Bit masks and their construction
- Register bit-field manipulation
- Portability considerations across different word sizes

Problem 2: GPIO Register Simulation

Description: Simulate a GPIO port with 32 pins. Implement functions to configure pins as input/output, write high/low to output pins, read input pins, and toggle output pins. Use memory-mapped register approach with a struct representing the GPIO registers.

What You Need to Learn:

- Memory-mapped I/O concept
- Struct packing and alignment
- Register bit definitions (MODER, ODR, IDR, BSRR)
- Hardware abstraction layer (HAL) design

Problem 3: Volatile Keyword Deep Dive

Description: Create a program that demonstrates why volatile is essential in embedded systems. Simulate a hardware timer that increments a counter. Show how compiler optimization breaks the code without volatile and works correctly with volatile. Examine the assembly output.

What You Need to Learn:

- Compiler optimization and its dangers in embedded
- When and why to use volatile
- Reading assembly output (objdump)
- Memory barriers and compiler barriers

Problem 4: Const Correctness in Embedded

Description: Implement a configuration table stored in flash memory using const. Create functions that accept const pointers where appropriate. Demonstrate the difference between const data, const pointers, and const pointer to const data.

What You Need to Learn:

- Const data placement in flash vs RAM
- Const correctness for function parameters
- ROM-able data structures
- Linker script sections (.rodata, .data, .bss)

Problem 5: Fixed-Point Arithmetic

Description: Implement fixed-point arithmetic operations (add, subtract, multiply, divide) for Q15.16 format. Create functions to convert between float and fixed-point. Compare precision and performance with floating-point operations.

What You Need to Learn:

- Fixed-point number representation

- Q-format notation (Q15.16, Q31, etc.)
- Overflow and saturation handling
- When to use fixed-point vs floating-point

Problem 6: Circular Buffer Implementation

Description: Implement a thread-safe circular buffer (ring buffer) for embedded systems. Support single producer/single consumer without locks using atomic operations. Include functions: init, put, get, is_empty, is_full, count.

What You Need to Learn:

- Circular buffer data structure
- Lock-free programming basics
- Memory ordering and atomic operations
- Power-of-two optimizations

Problem 7: State Machine Implementation

Description: Implement a finite state machine for a simple communication protocol parser. Use both switch-case approach and function pointer (table-driven) approach. The FSM should handle: IDLE, RECEIVING, PROCESSING, and ERROR states with transitions based on input events.

What You Need to Learn:

- Finite state machine theory
- Table-driven vs switch-driven FSMs
- Hierarchical state machines
- State machine frameworks (QP-nano, etc.)

Problem 8: Hardware Abstraction Layer Design

Description: Design a HAL for a UART peripheral. Create a layered architecture: register layer, driver layer, and application layer. The HAL should be portable across different

MCU families. Implement init, send, receive (blocking and non-blocking), and status functions.

What You Need to Learn:

- Layered software architecture
- Abstraction and portability
- Callback mechanisms
- Error handling in drivers

Problem 9: Memory Pool Allocator

Description: Implement a fixed-size memory pool allocator. Pre-allocate a block of memory and manage free blocks using a linked list. Implement: pool_init, pool_alloc, pool_free. Ensure no fragmentation and O(1) allocation/deallocation. Compare with malloc/free.

What You Need to Learn:

- Memory pool concepts
- Why malloc is problematic in embedded
- Deterministic memory allocation
- Fragmentation prevention

Problem 10: C++ for Embedded - Embedded-Friendly Classes

Description: Create C++ classes for embedded systems following best practices: no dynamic memory, no exceptions, minimal virtual functions. Implement a GPIO class, a Timer class, and a UART class using RAII principles. Ensure zero overhead abstraction.

What You Need to Learn:

- C++ subset for embedded (MISRA C++)
- RAII in embedded context
- Template metaprogramming for zero overhead

- Static polymorphism vs dynamic polymorphism

Phase 2: Microcontroller Architecture

Problems 11-20 | Focus: ARM Cortex-M, Registers, Clocks, Reset, Boot Process

What You Will Learn

Understanding the microcontroller's internal architecture is crucial for efficient embedded development. You will learn how the CPU interacts with peripherals, how clocks work, and what happens during power-up.

Problem 11: ARM Cortex-M Register Model

Description: Study and document the ARM Cortex-M4 register set: R0-R12, SP (MSP/PSP), LR, PC, PSR, and special registers (CONTROL, PRIMASK, FAULTMASK, BASEPRI). Write assembly code to manipulate these registers and observe their effects using a debugger simulation.

What You Need to Learn:

- ARM Cortex-M programmer's model
- General-purpose registers
- Special-purpose registers
- Privilege levels and thread/handler modes

Problem 12: Memory Map and Bus Architecture

Description: Create a memory map for a typical ARM Cortex-M4 microcontroller. Show the regions: Code (Flash), SRAM, Peripheral, External RAM, External Device, Private Peripheral Bus, and Vendor-specific. Calculate addresses for specific peripherals given base addresses.

What You Need to Learn:

- ARM memory map structure
- AHB, APB bus architecture
- Memory aliasing and bit-banding

- Vector table location and structure

Problem 13: Clock System Configuration

Description: Implement a clock configuration module for an STM32-like microcontroller. Configure the PLL to generate system clock from an 8MHz HSE crystal to achieve 168MHz system clock. Calculate all divider values and verify timing requirements.

What You Need to Learn:

- Clock sources: HSI, HSE, LSI, LSE
- PLL (Phase-Locked Loop) operation
- Clock dividers and prescalers
- Flash latency and wait states
- Clock security system (CSS)

Problem 14: Reset and Boot Process

Description: Trace through the boot process of an ARM Cortex-M microcontroller. Document what happens from power-on to main() execution: reset vector, initial SP value, SystemInit(), data section copying, BSS zeroing, and C runtime initialization.

What You Need to Learn:

- Reset types: Power-on, External, Watchdog, Software
- Startup code (startup_stm32xxxx.s)
- Vector table structure
- Linker script and sections
- C runtime initialization

Problem 15: Flash Memory Programming

Description: Implement functions to read, write, and erase flash memory. Handle flash programming algorithm: unlock, erase sector, program word, verify, lock. Implement

wear leveling for a simple configuration storage system.

What You Need to Learn:

- Flash memory organization (sectors/pages)
- Flash programming sequence
- Read-While-Write limitations
- Flash lifetime and wear leveling
- EEPROM emulation techniques

Problem 16: Option Bytes and Read Protection

Description: Implement a system to configure option bytes for read protection levels. Document the three levels of protection (Level 0, 1, 2) and their implications. Implement a secure bootloader that checks protection level before allowing firmware updates.

What You Need to Learn:

- Option bytes purpose and structure
- Read protection levels (RDP)
- Write protection
- Secure boot concepts
- Firmware protection strategies

Problem 17: Debug and Trace (SWD/JTAG)

Description: Set up a debugging environment using SWD (Serial Wire Debug). Implement printf-style debugging via SWO (Serial Wire Output). Configure DWT (Data Watchpoint and Trace) for data tracing. Understand how breakpoints are implemented.

What You Need to Learn:

- SWD protocol basics (2-wire vs 5-wire JTAG)
- ITM (Instrumentation Trace Macrocell)

- DWT and breakpoint implementation
- Semihosting vs retargeting
- Debug probe options (ST-Link, J-Link, DAPLink)

Problem 18: MPU (Memory Protection Unit) Configuration

Description: Configure the MPU to create protected memory regions. Set up regions for: code (execute-only), RAM (read-write), peripherals (read-write, no-execute), and stack (read-write with size limit). Handle MPU faults gracefully.

What You Need to Learn:

- MPU region attributes
- Access permissions (privileged vs unprivileged)
- Sub-region disable
- MPU fault handling
- RTOS integration with MPU

Problem 19: FPU (Floating Point Unit) Usage

Description: Enable and use the FPU on a Cortex-M4F. Implement DSP-style operations: FIR filter, FFT butterfly operation, and matrix multiplication. Compare performance with software floating-point. Handle lazy FPU context switching for interrupt safety.

What You Need to Learn:

- FPU register bank (S0-S31, D0-D15)
- Lazy stacking for interrupt latency
- FPU exception handling
- CMSIS-DSP library
- Fixed-point vs floating-point tradeoffs

Problem 20: Bootloader Design

Description: Design and implement a bootloader that can receive firmware updates via UART and program them to flash. Include: version checking, CRC verification, flash programming, and fallback to previous firmware on failure. Support in-application programming (IAP).

What You Need to Learn:

- Bootloader architecture
- Firmware update protocols (XMODEM, YMODEM, custom)
- Flash partitioning (bootloader vs application)
- Vector table relocation
- Rollback and recovery mechanisms

Phase 3: Memory Management

Problems 21-30 | Focus: Stack, Heap, BSS, Data, Linker Scripts, Memory Layout

What You Will Learn

Memory is the most constrained resource in embedded systems. You will learn how to analyze, optimize, and manage memory usage effectively.

Problem 21: Linker Script Deep Dive

Description: Write a complete linker script for an ARM Cortex-M microcontroller. Define MEMORY regions (FLASH, RAM), SECTIONS (.text, .data, .bss, .rodata, .stack, .heap), and symbols (_sdata, _edata, _sbss, _ebss, _estack). Understand the startup code interaction.

What You Need to Learn:

- Linker script syntax and structure
- MEMORY command and region attributes
- SECTIONS command and placement
- Linker-defined symbols
- Section alignment requirements

Problem 22: Stack Analysis and Overflow Detection

Description: Implement stack monitoring: initialize stack with pattern, calculate maximum stack usage, detect stack overflow. Create a stack canary mechanism. Analyze stack usage of your functions using compiler flags (-fstack-usage).

What You Need to Learn:

- Stack growth direction and overflow
- Stack canaries and guard pages
- Stack usage analysis tools

- Maximum stack depth calculation
- RTOS stack checking

Problem 23: Heap Management Without malloc

Description: Implement a deterministic heap allocator suitable for real-time systems. Use fixed-size block allocation or buddy allocation. Ensure O(1) allocation, no fragmentation, and thread safety. Compare memory overhead with standard malloc.

What You Need to Learn:

- Why malloc is non-deterministic
- Fixed-block allocator design
- Buddy allocation algorithm
- Heap integrity checking
- TLSF (Two-Level Segregated Fit) allocator

Problem 24: Memory Section Placement

Description: Place specific variables and functions in designated memory sections using attributes. Create a section for fast RAM (TCM), a section for battery-backed RAM, and a section for configuration data in flash. Access these sections correctly.

What You Need to Learn:

- __attribute__((section())) usage
- Tightly-Coupled Memory (TCM)
- Battery-backed RAM (BKP) usage
- Memory-mapped flash access
- Scatter file concepts

Problem 25: Const Data in Flash Optimization

Description: Optimize a lookup table to be stored in flash memory. Implement sine/cosine lookup tables, CRC tables, and font bitmaps in flash. Compare RAM usage

before and after optimization. Use const and PROGMEM equivalents.

What You Need to Learn:

- Flash vs RAM access tradeoffs
- Lookup table optimization
- Constexpr vs const
- Flash access acceleration (ART Accelerator)

Problem 26: Zero-Copy Buffer Design

Description: Design a zero-copy buffer system for DMA transfers. Implement a buffer pool where DMA can write directly and the application reads without copying. Handle buffer ownership, synchronization, and circular buffer management.

What You Need to Learn:

- Zero-copy architecture
- Buffer ownership semantics
- DMA buffer alignment requirements
- Cache coherency (clean/invalidate)

Problem 27: Memory-Mapped Peripheral Access

Description: Create a type-safe memory-mapped peripheral access layer. Use structs with volatile members to represent peripheral registers. Implement bit-field access macros that are type-safe and efficient. Compare with CMSIS-style access.

What You Need to Learn:

- Memory-mapped register access patterns
- Bit-field packing and alignment
- Type safety in register access
- CMSIS-Core register definitions

Problem 28: EEPROM Emulation in Flash

Description: Implement EEPROM emulation using flash memory. Design a wear-leveling algorithm that distributes writes across multiple flash pages. Handle page exhaustion, garbage collection, and power-fail safety.

What You Need to Learn:

- Flash vs EEPROM characteristics
- Wear leveling algorithms
- Page-based storage design
- Power-fail safe writes
- ST EEPROM emulation library

Problem 29: Memory Fault Injection and Hardening

Description: Implement memory fault detection mechanisms: ECC (Error Correcting Code) simulation, parity checking, and memory scrubbing. Create a fault injection framework to test error handling. Design recovery strategies for detected faults.

What You Need to Learn:

- Memory error types (SEU, MBU)
- Hamming codes for ECC
- Memory BIST (Built-In Self Test)
- Fault tolerance techniques

Problem 30: Static Memory Analysis Tools

Description: Use and interpret output from memory analysis tools: size, nm, objdump, readelf. Analyze a compiled binary to determine flash and RAM usage by function and data. Create a memory map visualization script.

What You Need to Learn:

- ELF file format basics

- Binutils (size, nm, objdump, readelf)
- Map file analysis
- Stack usage estimation
- Memory optimization strategies

Phase 4: Interrupts and Timers

Problems 31-40 | Focus: NVIC, ISR, Debouncing, PWM, Input Capture

What You Will Learn

Interrupts are the backbone of responsive embedded systems. You will master the Nested Vectored Interrupt Controller (NVIC), write efficient ISRs, and use timers for precise timing.

Problem 31: NVIC Configuration and Priorities

Description: Configure the NVIC for a set of peripherals with different priorities. Implement preemptive and sub-priority levels. Demonstrate priority inversion and how to avoid it. Handle nested interrupts correctly.

What You Need to Learn:

- NVIC architecture and registers
- Priority grouping (preemption vs sub-priority)
- Priority inversion scenarios
- Interrupt latency calculation

Problem 32: External Interrupt (EXTI) Handling

Description: Implement external interrupt handling for GPIO pins. Configure rising, falling, and both-edge triggers. Handle multiple EXTI lines. Implement a button press detection with debouncing in software and hardware.

What You Need to Learn:

- EXTI line mapping to GPIO
- SYSCFG and EXTI configuration
- Software debouncing techniques
- Hardware debouncing (RC circuits)

Problem 33: SysTick Timer for Delays

Description: Implement microsecond and millisecond delay functions using SysTick. Create both blocking and non-blocking (callback-based) delay mechanisms. Handle SysTick overflow and maintain accurate timing across long periods.

What You Need to Learn:

- SysTick timer architecture
- Reload value calculation
- Busy-wait vs interrupt-based delays
- Timing accuracy and jitter

Problem 34: General Purpose Timer (GPT) Mastery

Description: Configure a general-purpose timer for multiple modes: timebase, input capture, output compare, and PWM generation. Implement a servo motor control using PWM with 50Hz frequency and 1-2ms pulse width.

What You Need to Learn:

- Timer clock sources and prescalers
- Counter modes (up, down, center-aligned)
- Auto-reload register (ARR)
- Capture/Compare Register (CCR)
- PWM mode 1 vs mode 2

Problem 35: Input Capture for Frequency Measurement

Description: Implement frequency and duty cycle measurement using input capture mode. Measure an external signal with high accuracy. Handle overflow conditions and calculate period, frequency, and duty cycle.

What You Need to Learn:

- Input capture principles

- Edge detection and timestamping
- Timer overflow handling
- Frequency calculation algorithms

Problem 36: PWM Generation with Dead Time

Description: Generate complementary PWM signals with configurable dead time for H-bridge motor control. Implement break input for emergency stop. Configure advanced timer features for motor control applications.

What You Need to Learn:

- Advanced timers (TIM1, TIM8)
- Complementary outputs and dead time insertion
- Break input and fault handling
- Motor control applications (BLDC, PMSM)

Problem 37: Timer Synchronization and Master/Slave

Description: Configure timer synchronization: one timer as master trigger, others as slaves. Implement gated mode, trigger mode, and external clock mode. Create a synchronized multi-channel PWM system.

What You Need to Learn:

- Timer synchronization modes
- Master mode selection
- Slave mode configuration
- Trigger routing

Problem 38: Watchdog Timer Implementation

Description: Implement both independent watchdog (IWDG) and window watchdog (WWDG). Configure appropriate timeout periods. Implement a "kick" mechanism in the main loop. Handle watchdog reset detection and recovery.

What You Need to Learn:

- IWDG vs WWDG differences
- Watchdog clock sources
- Timeout calculation
- Window watchdog timing
- Reset source identification

Problem 39: Real-Time Clock (RTC) and Calendar

Description: Configure the RTC with battery backup domain. Implement calendar functions: get/set date and time, alarm configuration, and wakeup timer. Handle daylight saving time and timezone conversions.

What You Need to Learn:

- RTC architecture and clock sources (LSE, LSI)
- Backup domain and battery retention
- Calendar registers (BCD format)
- Alarm A and Alarm B
- Timestamp and tamper detection

Problem 40: DMA with Timer Triggers

Description: Configure DMA transfers triggered by timer events. Implement circular buffer for ADC data acquisition at precise intervals. Transfer data from memory to GPIO (bit-banging) using DMA without CPU intervention.

What You Need to Learn:

- DMA request mapping
- Timer DMA requests (update, compare)
- Circular DMA mode

- Double buffering

Phase 5: Communication Protocols

Problems 41-55 | Focus: UART, I2C, SPI, CAN, USB, Ethernet, Wireless

What You Will Learn

Communication is essential for embedded systems. You will master the most important protocols used in industry, from simple UART to complex CAN and wireless protocols.

Problem 41: UART Fundamentals and Implementation

Description: Implement a complete UART driver from scratch: initialization, transmit (polling and interrupt), receive (polling and interrupt), and error handling. Support configurable baud rates, data bits, parity, and stop bits. Implement ring buffer for RX and TX.

What You Need to Learn:

- UART frame format (start bit, data, parity, stop)
- Baud rate calculation and error
- Overrun, framing, and parity errors
- RS-232 vs TTL levels
- Flow control (RTS/CTS)

Problem 42: I2C Master Implementation

Description: Implement an I2C master driver supporting: start/stop conditions, 7-bit and 10-bit addressing, write and read operations, and repeated start. Communicate with a simulated EEPROM or sensor device. Handle arbitration and clock stretching.

What You Need to Learn:

- I2C bus specification (100kHz, 400kHz, 1MHz)
- Start, stop, and repeated start conditions
- ACK/NACK handling

- Clock stretching
- Bus arbitration (multi-master)

Problem 43: I2C Slave Implementation

Description: Implement an I2C slave device that responds to master requests. Create a register map that the master can read/write. Handle address matching, data transmission, and reception in interrupt-driven mode.

What You Need to Learn:

- I2C slave state machine
- Address matching and dual addressing
- Register map design
- Slave response timing

Problem 44: SPI Master with Multiple Slaves

Description: Implement an SPI master driver supporting multiple slave devices with individual chip selects. Support different SPI modes (0, 1, 2, 3) and configurable clock speeds. Communicate with multiple simulated sensors/displays.

What You Need to Learn:

- SPI signaling (MOSI, MISO, SCK, CS)
- Clock polarity and phase (CPOL, CPHA)
- Full-duplex communication
- Chip select management
- SPI bit ordering (MSB/LSB first)

Problem 45: SPI Flash Memory Interface

Description: Interface with an SPI flash memory (like W25Q128). Implement: read ID, read data, page program, sector erase, chip erase, and status register reading. Handle write-enable and busy-waiting. Implement a file system abstraction layer.

What You Need to Learn:

- SPI flash command set
- Page, sector, and block organization
- Write-enable and protection
- Status register polling
- Flash file system basics

Problem 46: CAN Bus Protocol Implementation

Description: Implement CAN 2.0B communication: configure bit timing, transmit and receive frames, handle filters and masks. Create a simple CANopen-like protocol layer with node IDs, PDOs, and SDOs. Simulate a multi-node network.

What You Need to Learn:

- CAN bus physical layer
- CAN frame format (standard vs extended)
- Bit timing and baud rate calculation
- Acceptance filters and masks
- Error handling and bus-off recovery
- CANopen and J1939 basics

Problem 47: USB Device (CDC ACM) Implementation

Description: Implement a USB CDC ACM (Virtual COM Port) device. Handle enumeration, standard requests, and CDC class-specific requests. Create a bridge between USB and UART. Support bulk IN/OUT endpoints for data transfer.

What You Need to Learn:

- USB architecture and descriptors
- Enumeration process

- CDC ACM class specification
- Endpoint configuration
- USB state machine

Problem 48: Ethernet and TCP/IP Stack

Description: Implement a lightweight TCP/IP stack (or use lwIP) on an Ethernet-enabled MCU. Support: ARP, IP, ICMP (ping), UDP, and TCP. Create a simple HTTP server that serves a status page. Handle multiple concurrent connections.

What You Need to Learn:

- Ethernet MAC and PHY
- lwIP configuration and integration
- Socket programming in embedded
- HTTP server implementation
- Memory constraints in TCP/IP

Problem 49: Modbus RTU and TCP

Description: Implement Modbus RTU (serial) and Modbus TCP (Ethernet) protocols. Support function codes: Read Coils, Read Discrete Inputs, Read Holding Registers, Read Input Registers, Write Single Coil, Write Single Register. Create both master and slave implementations.

What You Need to Learn:

- Modbus protocol specification
- RTU framing and CRC
- TCP MBAP header
- Exception responses
- Industrial communication standards

Problem 50: BLE (Bluetooth Low Energy) Basics

Description: Implement a BLE peripheral using a BLE module or integrated controller. Create a custom service with characteristics for sensor data. Implement advertising, connection, and notification/indication. Use a smartphone app to interact with the device.

What You Need to Learn:

- BLE protocol stack (GAP, GATT, ATT)
- Advertising and scanning
- Services and characteristics
- Notifications and indications
- Connection parameters

Problem 51: LoRa/LoRaWAN Communication

Description: Implement LoRa point-to-point communication. Configure spreading factor, bandwidth, and coding rate for range vs speed tradeoff. Implement basic LoRaWAN Class A device functionality: join procedure (OTAA), uplink, and downlink reception windows.

What You Need to Learn:

- LoRa modulation and parameters
- Link budget and range calculation
- LoRaWAN architecture
- Join procedures (OTAA vs ABP)
- Duty cycle and fair use

Problem 52: One-Wire Protocol (Dallas/Maxim)

Description: Implement the 1-Wire protocol for communicating with devices like DS18B20 temperature sensors. Handle reset/presence pulse, ROM commands (Search ROM, Match ROM, Skip ROM), and function commands. Implement ROM search

algorithm for multiple devices on the bus.

What You Need to Learn:

- 1-Wire signaling (reset, write, read)
- Time-slot timing
- ROM search algorithm
- CRC calculation
- Parasitic power mode

Problem 53: WS2812B LED Strip Control

Description: Implement control for WS2812B addressable LED strips. Generate precise timing signals (800kHz data rate, 0.4us/0.85us bit timing) using SPI or PWM+DMA. Create effects: color wipe, rainbow, chase, and fade.

What You Need to Learn:

- WS2812B timing requirements
- Bit-banging vs SPI/DMA approach
- Color representation (GRB format)
- Gamma correction
- Power consumption calculation

Problem 54: RS-485 Multi-Drop Network

Description: Implement an RS-485 multi-drop communication network. Handle half-duplex communication with DE/RE control. Implement a simple master-slave protocol with addressing. Handle bus termination and biasing.

What You Need to Learn:

- RS-485 electrical characteristics
- Half-duplex communication

- Bus termination and biasing
- Multi-drop topology
- Fail-safe biasing

Problem 55: Protocol Analyzer and Debugging

Description: Build a simple protocol analyzer that can sniff and decode UART, I2C, and SPI traffic. Display decoded data in human-readable format. Implement triggering on specific patterns. Use logic analyzer concepts for embedded debugging.

What You Need to Learn:

- Protocol decoding techniques
- Logic analyzer fundamentals
- Trigger conditions
- Signal integrity basics
- Debugging tools (Saleae, DSView)

Phase 6: RTOS and Concurrency

Problems 56-70 | Focus: FreeRTOS, Tasks, Semaphores, Queues, Scheduling

What You Will Learn

Real-Time Operating Systems are essential for complex embedded applications. You will learn task management, synchronization, and real-time scheduling concepts.

Problem 56: FreeRTOS Task Creation and Scheduling

Description: Set up FreeRTOS on your target MCU. Create multiple tasks with different priorities. Observe preemptive scheduling behavior. Implement periodic tasks using vTaskDelay and vTaskDelayUntil. Measure context switch time.

What You Need to Learn:

- FreeRTOS configuration (FreeRTOSConfig.h)
- Task creation and deletion
- Task states (running, ready, blocked, suspended)
- Preemptive vs cooperative scheduling
- Tick rate and timing

Problem 57: Binary and Counting Semaphores

Description: Implement task synchronization using binary semaphores for signaling and counting semaphores for resource counting. Create a producer-consumer pattern using semaphores. Handle priority inversion scenarios.

What You Need to Learn:

- Semaphore types and use cases
- Give/Take operations
- ISR-safe semaphore operations
- Priority inversion and inheritance

Problem 58: Mutex and Priority Inheritance

Description: Protect shared resources using mutexes. Demonstrate priority inheritance protocol in action. Create a scenario where priority inversion occurs and show how mutex with inheritance solves it. Measure the impact on worst-case execution time.

What You Need to Learn:

- Mutex vs binary semaphore
- Priority inheritance protocol
- Priority ceiling protocol
- Deadlock prevention

Problem 59: Queues for Inter-Task Communication

Description: Implement inter-task communication using FreeRTOS queues. Create multiple producers and consumers. Handle variable-sized messages using pointers. Implement a command queue pattern for processing requests sequentially.

What You Need to Learn:

- Queue creation and configuration
- Send and receive operations
- Blocking vs non-blocking operations
- Queue sets
- Stream buffers and message buffers

Problem 60: Event Groups for Synchronization

Description: Use event groups to synchronize multiple tasks waiting for different conditions. Implement a pattern where a task waits for multiple events (AND) or any of several events (OR). Handle event clearing and auto-clear semantics.

What You Need to Learn:

- Event group concepts

- Set bits and wait bits
- AND/OR synchronization
- Event group limitations

Problem 61: Software Timers

Description: Implement periodic and one-shot software timers using FreeRTOS timer API. Create a system with multiple timers at different periods. Handle timer callbacks efficiently. Compare software timers with hardware timers for different use cases.

What You Need to Learn:

- Timer creation (one-shot vs auto-reload)
- Timer ID and callback parameters
- Timer command queue
- Timer daemon task

Problem 62: Interrupt Service Routines with RTOS

Description: Write ISR-safe code that interacts with FreeRTOS. Use FromISR functions for semaphores and queues. Implement deferred interrupt processing using semaphores. Measure interrupt latency with and without RTOS.

What You Need to Learn:

- ISR-safe API (FromISR suffix)
- portYIELD_FROM_ISR macro
- Deferred interrupt processing
- Interrupt nesting with RTOS

Problem 63: Memory Management in RTOS

Description: Configure and use FreeRTOS heap implementations. Compare heap_1 through heap_5. Implement static allocation for critical tasks. Create a custom memory allocator using memory pools. Monitor stack usage of tasks.

What You Need to Learn:

- FreeRTOS heap implementations
- Static vs dynamic allocation
- Stack overflow detection
- uxTaskGetStackHighWaterMark
- Memory protection with MPU

Problem 64: Task Notifications

Description: Use task notifications as a lightweight alternative to semaphores and queues. Implement direct-to-task signaling. Use notifications for event groups, semaphores, and queue-like communication. Compare RAM usage with traditional primitives.

What You Need to Learn:

- Task notification value
- Notify and notify-wait operations
- Notification states
- Performance vs flexibility tradeoff

Problem 65: Real-Time Scheduling Analysis

Description: Perform Rate Monotonic Analysis (RMA) and Earliest Deadline First (EDF) analysis on a task set. Calculate CPU utilization and check schedulability. Measure actual response times and compare with theoretical predictions.

What You Need to Learn:

- Rate Monotonic Scheduling
- Utilization bound test (Liu & Layland)
- Response time analysis
- Priority assignment strategies

- Deadline miss detection

Problem 66: Low-Power Modes with RTOS

Description: Integrate low-power modes (sleep, stop, standby) with FreeRTOS. Implement tickless idle mode. Configure the MCU to enter low-power when idle and wake on interrupt. Measure power consumption in different modes.

What You Need to Learn:

- Tickless idle configuration
- configEXPECTED_IDLE_TIME_BEFORE_SLEEP
- PRE_SLEEP and POST_SLEEP processing
- Wakeup source configuration

Problem 67: Watchdog Integration with RTOS

Description: Integrate an independent watchdog with FreeRTOS. Create a watchdog task that monitors other tasks. Implement task health monitoring using heartbeats. Handle watchdog reset detection and recovery in a multi-task environment.

What You Need to Learn:

- Watchdog task design
- Task health monitoring
- Watchdog timeout calculation
- Recovery strategies

Problem 68: Debugging and Tracing

Description: Set up FreeRTOS+Trace or use SEGGER SystemView for real-time tracing. Configure trace hooks for task switches, queue operations, and semaphore operations. Analyze timing, CPU load, and task interactions visually.

What You Need to Learn:

- FreeRTOS trace macros

- SystemView integration
- Runtime statistics
- Task timing analysis

Problem 69: Multi-Core and AMP Systems

Description: Study asymmetric multiprocessing (AMP) on dual-core MCUs. Run FreeRTOS on one core and bare-metal or Linux on the other. Implement inter-core communication using shared memory and message passing. Handle resource sharing between cores.

What You Need to Learn:

- AMP vs SMP architectures
- Inter-core communication
- Hardware spinlocks
- Mailbox and message passing

Problem 70: Safety-Critical RTOS Concepts

Description: Study safety-critical RTOS requirements (IEC 61508, ISO 26262). Understand determinism, bounded execution, and fault tolerance. Explore certified RTOS options (SafeRTOS, PX5). Implement defensive programming techniques.

What You Need to Learn:

- Functional safety standards
- Deterministic behavior
- Fault detection and recovery
- Certified RTOS options
- ASIL levels in automotive

Phase 7: Low-Power Design

Problems 71-80 | Focus: Sleep Modes, Clock Gating, Power Measurement, Battery Life

What You Will Learn

Power consumption is critical for battery-operated devices. You will learn techniques to minimize power usage while maintaining functionality.

Problem 71: Sleep Mode Implementation

Description: Implement and measure all sleep modes on your MCU: Sleep, Deep Sleep (Stop), and Standby. Measure current consumption in each mode. Configure wakeup sources: external interrupt, RTC alarm, and watchdog. Compare wake-up times.

What You Need to Learn:

- MCU low-power modes
- WFI (Wait For Interrupt) and WFE
- Clock gating during sleep
- Wake-up time vs power tradeoff

Problem 72: Dynamic Voltage and Frequency Scaling

Description: Implement DVFS to reduce power consumption. Scale CPU frequency and voltage based on workload. Create a performance governor that automatically adjusts frequency. Measure power savings at different operating points.

What You Need to Learn:

- Voltage-frequency relationship
- PLL reconfiguration at runtime
- Flash wait states vs frequency
- Power governor algorithms

Problem 73: Peripheral Clock Gating

Description: Implement aggressive clock gating: disable clocks to unused peripherals. Create a peripheral manager that enables clocks on-demand and disables after use. Measure the impact on power consumption for various usage patterns.

What You Need to Learn:

- RCC (Reset and Clock Control)
- Peripheral clock enable/disable
- Clock domain considerations
- Bus matrix and clock distribution

Problem 74: GPIO Power Optimization

Description: Optimize GPIO configuration for low power: unused pins as analog input, pull-up/pull-down configuration, output speed settings. Implement a pin configuration audit tool. Measure leakage current with different configurations.

What You Need to Learn:

- GPIO modes and power consumption
- Floating pin dangers
- Output speed vs power
- I/O compensation cell

Problem 75: Battery Life Estimation

Description: Create a battery life calculator for your device. Profile current consumption in different operating modes. Calculate expected battery life based on duty cycle. Account for battery self-discharge and temperature effects.

What You Need to Learn:

- Battery capacity (mAh) and discharge curves
- Duty cycle calculation

- Average current consumption
- Battery chemistry effects

Problem 76: Energy Harvesting Interface

Description: Design an interface for energy harvesting sources (solar, thermal, vibration). Implement maximum power point tracking (MPPT) for solar panels. Create a power management system that switches between battery and harvested energy.

What You Need to Learn:

- Energy harvesting sources
- MPPT algorithms
- Power path management
- Supercapacitor usage

Problem 77: Brown-Out Detection and Handling

Description: Configure and test brown-out detection (BOD). Implement graceful shutdown when voltage drops. Save critical data before power loss. Design a system that can recover from brown-out conditions.

What You Need to Learn:

- Brown-out detection levels
- BOR (Brown-Out Reset) configuration
- PVD (Programmable Voltage Detector)
- Power-fail safe data storage

Problem 78: Ultra-Low-Power Sensor Reading

Description: Design a sensor reading system that minimizes power: wake up, take reading, transmit, sleep. Optimize wake-up time and sleep current. Implement adaptive sampling based on sensor value changes. Target sub-mA average current.

What You Need to Learn:

- Burst-mode operation
- Sensor warm-up time
- Adaptive sampling algorithms
- Radio power management

Problem 79: Power Measurement and Profiling

Description: Set up power measurement using a current sense resistor and ADC or dedicated power monitor IC. Profile power consumption of different code sections. Create a power profiler tool that shows current vs time with code correlation.

What You Need to Learn:

- Current measurement techniques
- INA219, INA226 power monitor ICs
- Profiling methodology
- Correlating code with power

Problem 80: Low-Power Design Checklist

Description: Create a comprehensive low-power design checklist for embedded projects. Include: clock configuration, peripheral management, GPIO settings, sleep mode usage, and measurement verification. Apply it to an existing project and document improvements.

What You Need to Learn:

- Systematic power optimization
- Design review process
- Power budgeting
- Documentation and validation

Phase 8: Hardware Interfacing

Problems 81-90 | Focus: ADC, DAC, Sensors, Motors, Displays, Storage

What You Will Learn

Connecting to the physical world is what makes embedded systems powerful. You will learn to interface with sensors, actuators, displays, and storage devices.

Problem 81: ADC Fundamentals and Configuration

Description: Configure the ADC for single conversion, continuous conversion, and scan mode. Implement different trigger sources: software, timer, and external. Measure conversion time and calculate maximum sampling rate. Handle multiple channels with DMA.

What You Need to Learn:

- ADC resolution and sampling rate
- Conversion modes
- Trigger sources
- Channel sequencing
- DMA integration

Problem 82: ADC Advanced Features

Description: Use ADC advanced features: oversampling for increased resolution, hardware averaging, analog watchdog for threshold detection, and injected channels for high-priority conversions. Implement temperature sensor reading using internal channel.

What You Need to Learn:

- Oversampling and decimation
- Analog watchdog

- Injected vs regular channels
- Internal temperature sensor
- Reference voltage

Problem 83: DAC Signal Generation

Description: Generate analog signals using DAC: sine wave, triangle wave, and arbitrary waveforms. Use DMA for continuous waveform generation without CPU intervention. Implement a simple audio player that plays PCM data through DAC.

What You Need to Learn:

- DAC resolution and settling time
- Waveform generation techniques
- DMA with DAC
- RC reconstruction filter

Problem 84: Temperature Sensor Interface

Description: Interface with different temperature sensors: analog (NTC thermistor, LM35), digital I2C (TMP102, SHT30), and one-wire (DS18B20). Implement linearization for NTC, calibration routines, and temperature compensation algorithms.

What You Need to Learn:

- Sensor types and characteristics
- NTC thermistor Steinhart-Hart equation
- Calibration techniques
- Temperature compensation

Problem 85: IMU Sensor Fusion (Accelerometer + Gyroscope)

Description: Interface with a 6-axis IMU (MPU6050 or similar) via I2C/SPI. Read raw accelerometer and gyroscope data. Implement complementary filter for angle estimation. Extend to Kalman filter for better performance. Calculate pitch and roll

angles.

What You Need to Learn:

- IMU data interpretation
- Complementary filter
- Kalman filter basics
- Sensor calibration (offset, scale)

Problem 86: DC Motor Control with PWM

Description: Control a DC motor using H-bridge driver with PWM speed control. Implement direction control, braking, and coasting. Add current sensing for overcurrent protection. Implement PID speed control for constant speed under varying load.

What You Need to Learn:

- H-bridge operation
- PWM speed control
- Current sensing (shunt resistor, Hall sensor)
- PID controller implementation

Problem 87: Stepper Motor Control

Description: Control a stepper motor in full-step, half-step, and microstepping modes. Implement acceleration and deceleration ramps. Create a motion profile generator for smooth movement. Add limit switch handling and homing routine.

What You Need to Learn:

- Stepper motor types (bipolar, unipolar)
- Stepping modes
- Acceleration ramping
- Motion profiles (trapezoidal, S-curve)

Problem 88: Character LCD and OLED Display Interface

Description: Interface with HD44780 character LCD (4-bit mode) and SSD1306 OLED (I2C/SPI). Implement text display, custom characters, and basic graphics. Create a menu system with navigation. Implement double buffering for smooth updates.

What You Need to Learn:

- HD44780 command set
- SSD1306 display RAM
- Framebuffer management
- Text rendering

Problem 89: SD Card Interface (SPI Mode)

Description: Interface with an SD card using SPI mode. Implement initialization sequence (CMD0, CMD8, ACMD41, CMD58, CMD16). Read and write sectors. Implement a simple FAT16/FAT32 file system layer for file creation, reading, and writing.

What You Need to Learn:

- SD card initialization
- SPI mode vs SD mode
- Block read/write
- FAT file system basics
- Chan's FatFs library

Problem 90: Touch Sensing (Capacitive and Resistive)

Description: Implement touch sensing: resistive touchscreen (ADC-based) and capacitive touch (TSC peripheral or external controller). Calibrate the touchscreen for accurate coordinate mapping. Implement gesture detection (tap, swipe, long press).

What You Need to Learn:

- Resistive touch principles

- Capacitive sensing basics
- Touchscreen calibration
- Gesture recognition

Phase 9: Edge AI and TinyML

Problems 91-100 | Focus: TensorFlow Lite Micro, Model Optimization, Neural Networks on MCU

What You Will Learn

Edge AI is revolutionizing embedded systems. You will learn to run machine learning models directly on microcontrollers, enabling intelligent devices that do not need cloud connectivity.

Problem 91: TensorFlow Lite Micro Setup

Description: Set up TensorFlow Lite Micro on your MCU. Build the library for your target. Run the Hello World example (sine wave prediction). Understand the memory requirements and performance implications. Measure inference time.

What You Need to Learn:

- TensorFlow Lite Micro architecture
- Model conversion (TFLite)
- Memory arena allocation
- Operator support on MCU

Problem 92: Keyword Spotting (KWS)

Description: Implement a keyword spotting system that recognizes spoken words (yes, no, up, down). Use pre-trained MicroNet model or train your own. Process audio from microphone, extract MFCC features, and run inference. Optimize for real-time performance.

What You Need to Learn:

- Audio preprocessing
- MFCC feature extraction
- Spectrogram generation

- CNN for audio classification

Problem 93: Gesture Recognition with IMU

Description: Build a gesture recognition system using accelerometer data. Collect training data for gestures (circle, wave, shake). Train a simple neural network (fully connected or LSTM). Deploy to MCU and recognize gestures in real-time.

What You Need to Learn:

- Motion signal processing
- Data collection and labeling
- LSTM for sequence classification
- Real-time inference

Problem 94: Person Detection with Vision

Description: Implement person detection using a camera module (OV7670 or similar) and a lightweight CNN (MobileNet SSD or person detection model from TensorFlow). Process image frames, run inference, and output detection results. Optimize for frame rate.

What You Need to Learn:

- Image preprocessing (resize, normalize)
- Quantized models (INT8)
- Object detection post-processing
- Frame rate optimization

Problem 95: Model Quantization and Optimization

Description: Quantize a floating-point model to INT8 for MCU deployment. Use post-training quantization and quantization-aware training. Compare accuracy before and after quantization. Apply pruning to reduce model size further.

What You Need to Learn:

- Quantization concepts
- Post-training quantization
- Quantization-aware training
- Pruning techniques
- Knowledge distillation

Problem 96: Anomaly Detection

Description: Implement anomaly detection for sensor data (vibration, current, temperature). Use autoencoder or one-class SVM. Train on normal data only. Detect anomalies in real-time and trigger alerts. Useful for predictive maintenance.

What You Need to Learn:

- Anomaly detection algorithms
- Autoencoder architecture
- Threshold selection
- Predictive maintenance concepts

Problem 97: Neural Network from Scratch on MCU

Description: Implement a simple neural network inference engine from scratch (no TFLite). Support fully connected layers with ReLU activation. Implement forward pass for a pre-trained MNIST digit classifier. Optimize using fixed-point arithmetic.

What You Need to Learn:

- Neural network inference
- Matrix multiplication optimization
- Fixed-point neural networks
- CMSIS-NN library

Problem 98: Federated Learning Concepts

Description: Study federated learning architecture for edge devices. Understand how models can be trained across distributed devices without sharing raw data. Implement a simple federated averaging algorithm. Explore privacy-preserving techniques.

What You Need to Learn:

- Federated learning architecture
- Federated averaging (FedAvg)
- Differential privacy
- Secure aggregation

Problem 99: Neural Architecture Search (NAS) for MCUs

Description: Explore Neural Architecture Search techniques to find optimal network architectures for MCU constraints. Use tools like TinyNAS or manually search the design space. Balance accuracy, latency, and memory usage for your target application.

What You Need to Learn:

- NAS concepts
- Constraint-aware search
- Latency prediction
- MCUNet and TinyNet architectures

Problem 100: End-to-End TinyML Application

Description: Build a complete TinyML application: data collection on device, model training (or transfer learning), deployment, and inference. Example applications: smart thermostat with occupancy detection, predictive maintenance sensor, or voice-controlled device. Document the entire ML pipeline.

What You Need to Learn:

- End-to-end ML workflow

- On-device data collection
- Model deployment pipeline
- OTA model updates
- Production considerations

Projects: Foundation to Expert Level

These 20 projects are designed to build your portfolio for EU companies. Each project targets skills that European employers value: automotive, industrial automation, IoT, and medical devices.

Hardware Recommendation

For learning (simulation only): QEMU, Renode, Proteus, STM32CubeIDE

For projects (recommended hardware purchase):

- STM32 Nucleo-F446RE or Nucleo-F429ZI (main development board)
- ST-Link V2 (debugger/programmer)
- Breadboard, jumper wires, resistors, capacitors, LEDs
- I2C OLED display (SSD1306, 128x64)
- Various sensors: DHT22 (temp/humidity), MPU6050 (IMU), HC-SR04 (ultrasonic)
- Actuators: SG90 servo, DC motor with driver (L298N), stepper motor
- Communication modules: CAN transceiver (MCP2551), ESP8266 (WiFi), nRF24L01 (wireless)
- Power supply: 5V/3.3V breadboard supply, LiPo battery with charger
- Logic analyzer (optional but highly recommended): Saleae Logic clone (~$10)
- Oscilloscope (optional): DSO138 kit (~$30) or similar

Projects 1-5: Foundation Level

Project 1: Digital Clock with Alarm

Description: Build a digital clock using RTC peripheral. Display time on LCD or OLED. Implement alarm functionality with buzzer. Add buttons for time setting. Use low-power modes when idle. Store alarm settings in backup RAM.

Skills Demonstrated:

- RTC configuration and calendar functions
- GPIO input (buttons) with debouncing
- Timer-based PWM for buzzer
- Display interface (I2C/SPI)
- Low-power design

Hardware Required: Nucleo board, OLED display, 3 buttons, buzzer, breadboard

EU Relevance: Consumer electronics, IoT devices

Project 2: Temperature Monitoring System

Description: Create a multi-sensor temperature monitoring system. Read from 3+ temperature sensors (analog and digital). Display readings on LCD. Log data to SD card with timestamps. Implement high-temperature alarm. Send data via UART to PC for plotting.

Skills Demonstrated:

- ADC for analog sensors
- I2C for digital sensors
- SD card interface (SPI)
- RTC for timestamps
- UART communication

Hardware Required: Nucleo board, LM35 or NTC, I2C temp sensor, SD card module, LCD

EU Relevance: Industrial monitoring, HVAC systems, cold chain

Project 3: Serial Data Logger

Description: Build a standalone serial data logger that records data from external devices. Two UART ports: one for receiving data, one for configuration. Store data to SD card in CSV format. Implement circular buffer for burst handling. Configurable baud rate and frame format via CLI.

Skills Demonstrated:

- Dual UART handling
- Circular buffer implementation
- SD card file operations
- Command-line interface
- Data parsing and formatting

Hardware Required: Nucleo board, SD card module, USB-to-UART adapter for testing

EU Relevance: Industrial data acquisition, telemetry

Project 4: Motor Speed Controller

Description: Build a closed-loop DC motor speed controller. Read speed from encoder or tachometer. Implement PID controller for precise speed regulation. Control motor via PWM. Provide user interface (potentiometer or buttons) for speed setting. Display actual and target speed on LCD.

Skills Demonstrated:

- PWM motor control
- Encoder/tachometer reading
- PID controller implementation

- Real-time control loop
- User interface

Hardware Required: Nucleo board, DC motor, motor driver (L298N or DRV8833), encoder, LCD

EU Relevance: Robotics, CNC, industrial drives

Project 5: Home Automation Hub

Description: Create a central hub for home automation. Connect multiple sensors (temperature, motion, light). Control relays for lights and appliances. Implement a simple menu system on LCD for local control. Log events to SD card. Use RTC for scheduling (turn on lights at sunset).

Skills Demonstrated:

- Multiple sensor integration
- Relay control
- Menu system design
- Event logging
- Time-based scheduling

Hardware Required: Nucleo board, multiple sensors, relay module, LCD, SD card

EU Relevance: Smart home, building automation

Projects 6-10: Intermediate Level

Project 6: CAN Bus Vehicle Diagnostic Tool

Description: Build a CAN bus diagnostic tool. Connect to vehicle OBD-II port or simulated CAN network. Decode and display CAN messages. Implement request/response for diagnostic trouble codes (DTCs). Log CAN traffic to SD card. Provide real-time display of vehicle parameters (RPM, speed, temperature).

Skills Demonstrated:

- CAN protocol implementation
- OBD-II protocol
- Message filtering and decoding
- Data logging
- Real-time data display

Hardware Required: Nucleo board with CAN, CAN transceiver (MCP2551), OBD-II cable or second CAN node for simulation

EU Relevance: Automotive (Bosch, Continental, VW, BMW), high demand in Germany

Project 7: FreeRTOS-Based Sensor Gateway

Description: Build a sensor gateway using FreeRTOS. Multiple tasks: sensor reading (I2C, SPI, ADC), data processing, LCD display, SD card logging, and UART communication. Use queues for inter-task communication. Implement priority-based scheduling. Add watchdog for system reliability.

Skills Demonstrated:

- FreeRTOS task management
- Inter-task communication (queues, semaphores)
- Resource sharing protection
- Priority-based design

- Watchdog integration

Hardware Required: Nucleo board, multiple sensors, SD card, LCD

EU Relevance: Industrial IoT, gateway devices

Project 8: Bootloader with Firmware Update

Description: Design and implement a complete bootloader system. Bootloader occupies first flash sector, application in remaining sectors. Receive firmware updates via UART (YMODEM protocol or custom). Verify CRC before programming. Support rollback to previous version. Jump to application after verification.

Skills Demonstrated:

- Bootloader architecture
- Flash programming
- Protocol implementation (YMODEM)
- CRC verification
- Vector table relocation

Hardware Required: Nucleo board, USB-to-UART adapter

EU Relevance: All embedded products requiring OTA updates

Project 9: Battery Management System Monitor

Description: Build a BMS monitoring system for Li-ion battery pack. Measure cell voltages (using ADC multiplexer), pack current (shunt resistor), and temperature. Calculate State of Charge (SOC) using coulomb counting. Implement balancing control logic. Display status on LCD. Log data for analysis.

Skills Demonstrated:

- Precision ADC measurement
- Current sensing
- SOC estimation algorithms

- Cell balancing concepts
- Safety monitoring

Hardware Required: Nucleo board, LiPo battery, current sense resistor, temp sensors, LCD

EU Relevance:EV industry (Tesla, VW, BMW), energy storage

Project 10: Industrial Modbus RTU/TCP Gateway

Description: Create a protocol gateway that translates between Modbus RTU (serial) and Modbus TCP (Ethernet). Support multiple Modbus function codes. Implement concurrent connections. Add web interface for configuration (optional). Log all transactions.

Skills Demonstrated:

- Modbus RTU and TCP protocols
- Protocol translation
- lwIP TCP/IP stack
- Concurrent connection handling
- Transaction logging

Hardware Required: Nucleo board with Ethernet, RS-485 transceiver

EU Relevance: Industrial automation (Siemens, Schneider, ABB)

Projects 11-15: Advanced Level

Project 11: Safety-Critical Motor Controller (IEC 61508 Concepts)

Description: Design a motor controller with safety considerations. Implement dual-channel emergency stop. Add watchdog and self-diagnostics. Use MPU for memory protection. Document safety requirements and failure modes. Follow defensive programming practices.

Skills Demonstrated:

- Safety-critical design
- Redundancy and diagnostics
- MPU usage
- Defensive programming
- Documentation practices

Hardware Required: Nucleo board, motor, driver, dual E-stop buttons

EU Relevance: Functional safety (automotive, industrial machinery)

Project 12: USB Audio Device with DSP

Description: Implement a USB audio class device. Receive audio from PC via USB. Apply DSP effects (volume control, equalizer, echo). Output via DAC or I2S codec. Support different sample rates. Use DMA for efficient data transfer.

Skills Demonstrated:

- USB Audio Class
- I2S interface
- DSP algorithms
- DMA audio streaming
- Real-time processing

Hardware Required: Nucleo board, I2S audio codec (WM8731 or similar)

EU Relevance: Audio equipment, pro audio industry

Project 13: LoRaWAN Environmental Sensor Node

Description: Build a battery-powered LoRaWAN sensor node. Read temperature, humidity, and air quality sensors. Implement LoRaWAN Class A protocol stack (use library like LMIC). Join network via OTAA. Send data periodically. Implement adaptive data rate. Target 1+ year battery life.

Skills Demonstrated:

- LoRa/LoRaWAN protocols
- Low-power design
- OTAA join procedure
- Adaptive data rate
- Battery life optimization

Hardware Required: Nucleo board, LoRa module (SX1276), sensors, LiPo battery

EU Relevance: Smart city, agriculture, environmental monitoring

Project 14: Inertial Navigation System

Description: Build an inertial navigation system using IMU (MPU9250 with magnetometer). Implement sensor fusion (complementary or Kalman filter) for attitude estimation. Calculate position by integrating acceleration (dead reckoning). Compensate for drift. Display orientation and position on PC via UART.

Skills Demonstrated:

- IMU data fusion
- Kalman filter implementation
- Attitude estimation
- Dead reckoning

- Drift compensation

Hardware Required: Nucleo board, MPU9250 9-DOF IMU

EU Relevance: Drones, robotics, autonomous vehicles

Project 15: TCP/IP Network Stack with Custom Protocol

Description: Implement a lightweight TCP/IP stack or deeply customize lwIP. Add a custom application-layer protocol for device management. Support device discovery, configuration, firmware update, and telemetry. Implement secure communication (optional: TLS).

Skills Demonstrated:

- TCP/IP stack internals
- Protocol design
- Socket programming
- Network security
- Device management

Hardware Required: Nucleo board with Ethernet

EU Relevance: IoT, industrial networking

Projects 16-20: Expert Level

Project 16: Edge AI Voice Controller

Description: Build a voice-controlled device using TinyML. Implement keyword spotting for command recognition (lights on, temperature, etc.). Use TensorFlow Lite Micro. Process audio from microphone in real-time. Execute commands based on recognized keywords. Display status on LCD.

Skills Demonstrated:

- TensorFlow Lite Micro
- Audio preprocessing
- Keyword spotting
- Real-time inference
- Command execution

Hardware Required: Nucleo-F4/F7 (higher performance), I2S microphone, LCD

EU Relevance: Edge AI, smart home, voice interfaces - cutting edge

Project 17: Predictive Maintenance Sensor

Description: Build a vibration analysis system for predictive maintenance. Collect vibration data from accelerometer. Train anomaly detection model (autoencoder) to detect bearing faults. Run inference on MCU. Send alerts when anomalies detected. Log vibration spectra for analysis.

Skills Demonstrated:

- Vibration analysis
- FFT on embedded
- Anomaly detection
- Machine learning on MCU
- Industrial IoT

Hardware Required: Nucleo board, ADXL355 or similar accelerometer, WiFi/Ethernet module

EU Relevance: Industry 4.0, predictive maintenance - very high demand

Project 18: Automotive ECU Simulator

Description: Build a multi-protocol automotive ECU simulator. Support CAN, LIN, and K-Line protocols. Simulate engine ECU responses to diagnostic requests. Implement OBD-II PIDs. Generate realistic sensor data. Use for testing diagnostic tools. Document all protocols implemented.

Skills Demonstrated:

- Multiple automotive protocols
- OBD-II implementation
- Protocol simulation
- Diagnostic services
- Automotive domain knowledge

Hardware Required: Nucleo board, CAN transceiver, LIN transceiver

EU Relevance: Automotive industry - Germany, France, Italy

Project 19: Medical Device Controller (Concept)

Description: Design a controller for a medical device (infusion pump or ventilator concept). Implement safety-critical features: dual-channel sensors, watchdog, alarms. Use state machine for control logic. Document risk analysis (FMEA). Follow IEC 62304 concepts. Note: This is a learning project, not for actual medical use.

Skills Demonstrated:

- Safety-critical design
- State machine implementation
- Risk analysis

- Medical standards awareness
- Documentation

Hardware Required: Nucleo board, sensors, actuators (pump motor or servo)

EU Relevance: Medical devices - Philips, Siemens Healthineers, Medtronic

Project 20: Complete IoT Platform Node

Description: Build a production-ready IoT sensor node. Multiple sensors (temp, humidity, light, motion). Connectivity: WiFi or LoRaWAN. Local processing with FreeRTOS. Edge ML for local decision making. OTA firmware updates. Cloud integration (MQTT to AWS IoT or Azure IoT). Power management for battery operation. Complete documentation and test suite.

Skills Demonstrated:

- Full-stack embedded development
- RTOS integration
- Wireless communication
- Cloud connectivity
- OTA updates
- Power management
- Edge ML

Hardware Required: Nucleo board, multiple sensors, WiFi module (ESP8266/ESP32) or LoRa module

EU Relevance: Complete IoT solution - applicable to any industry

EU Job Market Guide

The European embedded systems job market is strong and growing. This section provides guidance on finding positions, understanding requirements, and succeeding in EU tech companies.

Key EU Markets for Embedded Engineers

Germany (Largest Market)

- Automotive: BMW, Mercedes-Benz, VW, Audi, Porsche, Bosch, Continental
- Industrial: Siemens, Festo, Phoenix Contact, Beckhoff
- Medical: Siemens Healthineers, Drager, Fresenius
- Average Salary: EUR 50,000-85,000 (junior to senior)
- Language: English often sufficient; German is a plus

Netherlands

- Semiconductors: NXP, ASML, Philips
- Automotive: VDL, DAF
- High-tech systems: Many innovative startups
- Average Salary: EUR 45,000-80,000
- Language: English widely accepted

France

- Aerospace: Airbus, Thales, Safran
- Automotive: Renault, PSA/Stellantis, Valeo
- Energy: Schneider Electric
- Average Salary: EUR 40,000-75,000
- Language: French often required

Switzerland

- High precision: ABB, Maxon, u-blox
- Medical: Roche, Sonova
- Average Salary: CHF 90,000-150,000 (highest in Europe)
- Language: German/French/English depending on region

Other Notable Markets

- Sweden: Volvo, Ericsson, Atlas Copco
- Italy: Ferrari, Lamborghini, STMicroelectronics
- UK: ARM (now owned by SoftBank), Raspberry Pi, many startups
- Austria: AVL List (automotive), Kapsch
- Denmark: Danfoss, Grundfos

Skills in High Demand

Technical Skills (Ranked by Demand)

0. **C/C++ for Embedded**: Essential - master it completely
1. **ARM Cortex-M**: Most common architecture in EU
2. **RTOS (FreeRTOS, ThreadX, Zephyr)**: At least one
3. **Communication Protocols**: CAN, LIN, Ethernet, SPI, I2C
4. **Automotive Standards**: AUTOSAR, ISO 26262, ASPICE
5. **Linux Embedded**: Yocto, Buildroot, device drivers
6. **Python**: For testing, scripting, ML integration
7. **Version Control**: Git, with good practices
8. **CI/CD**: Jenkins, GitLab CI for embedded
9. **Edge AI/TinyML**: Emerging high-demand skill

Soft Skills

- English proficiency (minimum B2, ideally C1)
- Documentation skills (very important in EU)
- Agile/Scrum methodology experience
- Requirements engineering understanding
- Testing mindset (unit tests, integration tests)

Certifications That Help

- AUTOSAR - Critical for automotive (expensive but valuable)
- ISO 26262 - Functional safety for automotive
- IEC 61508 - Industrial functional safety
- ARM Accredited Engineer - Shows ARM expertise
- FreeRTOS + AWS IoT - Good for IoT roles

Job Application Strategy

Building Your EU-Friendly Portfolio

- GitHub with clean, documented code
- Projects demonstrating safety-critical thinking
- Documentation in English (clear, professional)
- Blog posts or articles about your projects
- Video demonstrations of working projects
- Contributions to open-source embedded projects

Where to Apply

- LinkedIn - Primary platform for EU tech jobs

- Company career pages directly
- StepStone, Indeed, Monster (EU versions)
- Honeypot.io (reverse recruiting for developers)
- Stack Overflow Jobs
- Relocation-friendly platforms: Relocate.me, Landing.jobs

CV/Resume Tips for EU Market

- Use Europass format or clear, professional template
- Include photo (common in Germany, optional elsewhere)
- List languages with proficiency levels (CEFR)
- Highlight relevant standards (AUTOSAR, ISO 26262)
- Include links to GitHub, portfolio, LinkedIn
- Keep to 2 pages maximum
- Translate to local language if applying to non-English companies

Interview Preparation

- Technical: C pointers, bit manipulation, RTOS concepts
- Coding test: Often on paper or whiteboard
- System design: Architecture questions for embedded systems
- Behavioral: STAR method for answering
- Company research: Know their products and technologies
- Questions to ask: Team structure, development process, growth opportunities

Visa and Relocation

EU Blue Card

- For highly skilled non-EU workers
- Requires: University degree + job offer with minimum salary threshold
- Germany: EUR 45,300+ (2024) for shortage occupations (IT/engineering)
- Benefits: Fast-track permanent residency, family reunification

Job Seeker Visa

- Germany offers 6-month visa to search for jobs
- Requires: Qualifications + funds to support yourself

Company-Sponsored Relocation

- Many large companies offer relocation packages
- May include: Flight, temporary housing, relocation agent
- Always ask about relocation support in interviews

Salary Expectations (2024)

Note: Varies by country, company size, and your experience

- Junior (0-2 years): EUR 35,000-55,000
- Mid-level (3-5 years): EUR 50,000-75,000
- Senior (5+ years): EUR 70,000-100,000+
- Principal/Architect: EUR 90,000-130,000+
- Switzerland: Add 50-80% to above figures

Continuous Learning

- Follow ARM Community and Embedded.com

- Read Embedded Systems Design magazine
- Join LinkedIn groups for embedded professionals
- Attend embedded world exhibition (Nuremberg, Germany)
- Take online courses on new technologies
- Contribute to Zephyr RTOS or other open-source projects

Final Words

You now have a complete roadmap to becoming an exceptional embedded systems engineer, ready for the EU job market. This guide contains 100 carefully designed problems and 20 projects that cover the full spectrum of embedded engineering.

Your Path Forward

10. **Start with Simulations**: Use QEMU, Renode, or STM32CubeIDE to practice without hardware
11. **Master the Fundamentals**: Complete all 100 problems - they build upon each other
12. **Build the Projects**: Start with foundation projects, progress to expert level
13. **Document Everything**: Keep an engineering notebook, blog about your learning
14. **Share Your Work**: GitHub portfolio is essential for EU job applications
15. **Invest in Hardware**: Buy recommended hardware for hands-on experience
16. **Network**: Join embedded communities, attend meetups and conferences
17. **Never Stop Learning**: Embedded systems evolve - stay current with new technologies

Remember

The embedded engineers who get hired at top EU companies are those who can demonstrate practical skills through real projects. Companies like Bosch, Siemens, and BMW receive thousands of applications - your portfolio is what will set you apart.

Every problem you solve, every project you complete, brings you closer to your goal. The journey may be challenging, but the reward - a career building the technology that powers our world - is worth it.

Go build amazing things. The embedded world is waiting for you.

Embedded Systems

Engineering Mastery Guide

100 Problems | 20 Projects | Edge AI & TinyML

EU Job Market Ready